

DEEPL - Lab 5

Recurrent Networks: Sequence to Sequence (seq2seq)

Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowarski - Laure Soulier

2025-2026

Introduction (brief, see lecture)

In this third and final lab on RNNs, we will study two "sequence to sequence" (Seq2Seq) tasks :

- labeling each element of the sequence (the number of input and output elements are the same) ;
- generating a sequence from a latent state. This latent state represents the input data, and depends on the task : sentence representation (translation/question answering), image representation (image captioning), etc.

Compared to previous labs, three new features :

1. use of RNNs defined by torch that take as input a **PackedSequence** and return similar structures as output (they can also be used with standard batches).
2. use of variable-size encoding (*wordpiece*)
3. use of *curriculum learning* for training

1 Tagging

In this exercise, we will focus on the Part-Of-Speech parsing task, which consists of assigning a grammatical category/nature to each word. We will use the GSD dataset (see code skeleton for downloading). Each example is a sentence, already segmented into tokens, for which we will focus on the raw word (**form**) and the tag (**upostag**).

The following code is provided :

- the **Vocabulary** class which manages a vocabulary (correspondence between a token and its index) ; it will be used both to index words and to index tags ;
- the **TaggingDataset** class which allows building a dataset where each item is a pair (token, tag), the tokens being the words of a sentence and the **tags** the associated categories ;
- the **collate_fn** collate function of the **DataLoader** which uses the **pad_sequence** function to form sequences of the same size (see below).

Padding as seen in the previous lab can be done with PyTorch's `pad_sequence` function. The `ignore_index` parameter of cost functions allows indicating the padding character index to avoid taking padding into account in the cost calculation (like the mask from the last session). However, completing sequences this way has a computational cost that can be very high because many unnecessary operations will be performed (during network feeding and also during loss calculation). PyTorch allows limiting this side effect by using so-called "packed" sequences : sequences are flattened and all aligned on the same dimension, the actual length of sequences is stored jointly, and only necessary operations will be performed. The `pack_sequence` function applied to a list of tensors allows obtaining a `PackedSequence`, a tuple that contains the batch on one side and the length of each batch element on the other. To obtain this flattened tensor, you can also use the `pack_padded_sequence` function on an already padded batch. The inverse operation is `pad_packed_sequence` which allows obtaining a padded tensor from a flattened tensor.

Question 1

Implement a `seq2seq` model for tagging and the training loop using PyTorch's LSTM module and sequence padding (packing is a bonus).

To account for the problem of unknown words (OOV, *Out of Vocabulary*) that may appear at test time^a, we can randomly replace words in the training examples with OOV words (using a special token - [OOV]). Implement and compare the results.

To visualize the results, display for a given input sentence the associated sequence of tags.

^a. Unlike previous labs, we are working with tokens and no longer with characters. It is very likely that some words will only appear during testing. These words are called OOV.

2 Translation

For the translation task, we will use two RNNs :

- an encoder that is responsible for producing a hidden state after reading the sequence to be translated
- a decoder, which from the hidden state, will generate the translated sentence.

In addition to the *EOS* (*End of Sequence*) token, you will need a special *SOS* (*Start of Sequence*) token which will be the first token given as input to the decoder (in addition to the hidden state) from which the sentence is translated.

There are two ways to train the decoder :

- constrained mode (or *teacher forcing*) where the target sentence is passed to the decoder and at each time step a word from the target sentence is considered : the

- generation is guided and allows precisely correcting each produced latent state ;
- unconstrained mode where the target sentence is not considered during the iterative generation of the translation : it is the word corresponding to the maximum probability of the previous time step's latent state decoding that is introduced at each time step (or a random draw from this distribution) ; this mode consists of generating as if you were in inference and then correcting once the entire sentence has been generated.

Unconstrained mode is more difficult than constrained mode : an error when predicting a word will greatly disturb the generation and backpropagation will not be very effective for the rest of the sequence. However, it allows better generalization and avoids rote learning. Intuitively, one should start training using constrained mode to properly initialize the decoder and then gradually switch to unconstrained mode. This type of procedure is called Curriculum learning.

Question 2

In `tp5-traduction.py`, implement the encoder-decoder. Use GRUs in both cases and the following architectures :

- encoder : an embedding of the source vocabulary followed by a GRU
- decoder : an embedding of the target vocabulary, followed by a GRU and a linear network for decoding the latent state (and a softmax to finish)

In the decoder, you will need a `generate(hidden, lenseq=None)` method that from a hidden state `hidden` (and the SOS token as input) produces a sequence until the length `lenseq` is reached or until the EOS token is generated.

Implement the training loop using a simple Curriculum learning strategy that consists of randomly drawing uniformly between constrained mode and unconstrained mode for each mini-batch^a. You can pass the expected length of target sentences during generation in the case of unconstrained mode.

Train your model while keeping a test set to verify that you are not over/under-fitting. Use (at test time) the generation methods from the previous lab to visualize the proposed translations.

^a. In the original paper, the choice between the two modes is made according to a decreasing probability for constrained mode and this choice is made at each time step, not for the entire minibatch. As a bonus, you can improve the strategy and compare the results.

3 Preprocessing and efficient text segmentation

Text preprocessing relies on a segmentation step where the text is divided into linguistic units. For a long time, the chosen level was the word (= alphanumeric string surrounded by spaces) ; in recent years, alternatives have been (re)explored with new neural models.

One of the most efficient segmentations currently is the variable n-gram splitting (*subword*

units) popularized by Byte-Pair Encoding (BPE) in 2016. These segmentations have the advantage of having a fixed vocabulary size that best covers the dataset, and avoids the problem of unknown words.

For example, "You shoulda got David Carr of Third Day to do it" will be segmented as "_You", "_should", "a", "_got", "_D", "av", "id", "_C", "ar", "r", "_of", "_Th", "ir", "d", "_Day", "_to", "_do", "_it" where frequent sequences (e.g., You, should) are extracted directly while less frequent sequences (e.g., David, Carr) are segmented into several parts.

You will use the sentencepiece library (don't forget to update the python modules if you are not working on a university machine). Unknown <unk>, BOS (<s>) and EOS (</s>) tokens are predefined, but you can add others with `user_defined_symbols` :

```
import sentencepiece as spm
spm.SentencePieceTrainer.train(
    input='FILEPATH',
    model_prefix='MODEL_OUTPUT_PATH',
    vocab_size=1000, user_defined_symbols=[]
)
```

To segment a sentence, you can use the following instructions :

```
import sentencepiece as spm
s = spm.SentencePieceProcessor(model_file='MODEL_PATH.model')

# Returns token identifiers
ids = s.encode('New York', out_type=int)

# Returns the corresponding string
sp.decode(ids)

# Returns the tokens
s.encode('New York', out_type=str)
```

Question 3

Create the segmentation model for the dataset then change the `Dataset` to use variable segmentation. Once segmentation is in place, tackle the problem of training a translation model. You can compare results with and without segmentation.